

2026 US LLVM Developers' Meeting

27 October 2026

How to build a Python DSL for MLIR

Extracting DkgDSL's reusable core into mlir dsl

Guray Ozen

Principal Compiler Engineer, NVIDIA

Amir Tavakkoli

Why a Python DSL for MLIR?

Four useful things to build with mlir dsl.

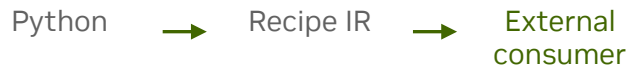
01 Test an internal dialect

Generate the intermediate IR directly; test its lowering in isolation.



02 Generate a recipe dialect

Use Python to author IR for a consumer outside MLIR.



03 Reuse the building blocks

Bring JIT, caching, and environment management to your own DSL.



04 Build a complete language

A production example: NVIDIA CuTe DSL / CUTLASS Python.



[NVIDIA CuTe DSL documentation](#) 

Use as much of the infrastructure as your dialect needs.

What should a Python MLIR DSL provide?

Extract the common machinery from DkgDSL; keep target choices in sub-DSLs.

mlir dsl · reusable BaseDSL infrastructure

Types + inference

AST + value trees

AOT + JIT + cache

Compile + execute

Env + profiling

Errors + remarks

Arrays + adapters

FFI + export hooks



Your sub-DSL

Domain types

Capture callbacks

Dialect + lowering

DkgDSL base_dsl → mlir dsl core; dialect operations and device policy stay downstream.

Design principles

Keep the language familiar.

01 Don't make me learn a new language.

Reuse Python objects, imports, libraries, and metaprogramming.

02 Use Python as-is as the host.

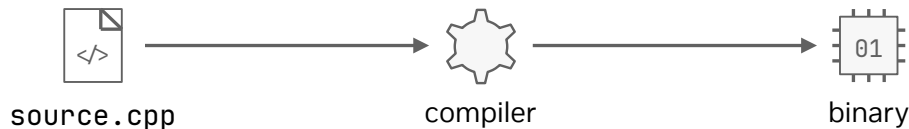
Keep Python's meaning; expose MLIR through a DSL namespace.

This leads to **multi-stage programming**.

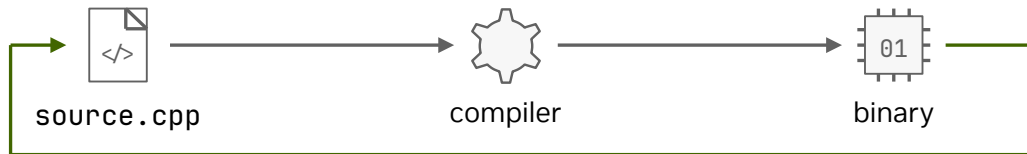
What is multi-stage programming?

Run a program to generate the next program.

Traditional compiler



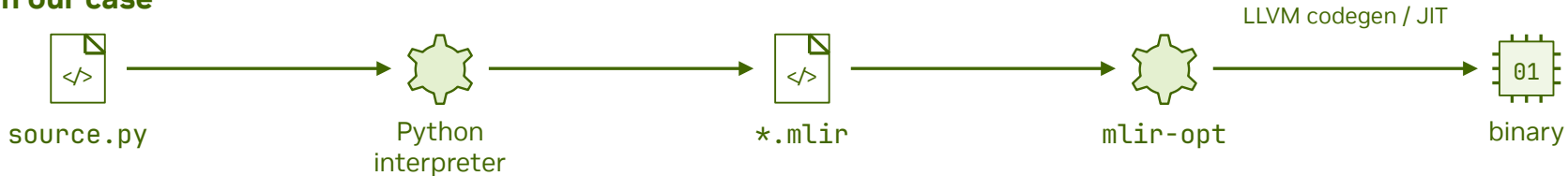
Multi-stage compilers



Run binary → generate source for the next stage

... Repeat for further stages.

In our case



Why multi-stage programming?

Keep Python's expressiveness. Make MLIR generation explicit.

01 PYTHON AS THE META-PROGRAM

Use all of Python.

Objects, imports, libraries, and metaprogramming are available to build the program.

02 MLIR THROUGH A NAMESPACE

Make capture explicit.

Put MLIR types and builders in a DSL namespace. Choose what to capture through that API.

Inside @m.jit, with import mlir.dsl as m

`range(4)`



Python loop

Runs during generation

`m.range(4)`



`scf.for`

Captured in MLIR

Python builds the program. The DSL namespace makes MLIR capture explicit.

How to build a Python DSL for MLIR

Run Python. Let DSL values and builders generate the target program.

Python

Python + a DSL namespace

```
import mlir.dsl as m

@m.jit
def scale(x: m.Int32, factor):
    return x * factor

scale(4, 3)
```

Objects · imports · metaprogramming

Trace: execute Python

Meta

factor = 3
Compute in Python

Staged

x = SSA value
Emit MLIR operations

MLIR

Generated MLIR · body excerpt

```
// x is a runtime i32
%c3 = arith.constant 3 : i32
%r = arith.muli %x, %c3 : i32
llvm.return %r : i32
```

Compile

Test

Consume IR

The interpreter runs the generator; your dialect defines the generated program.

Language semantics: two kinds of values

Python is the metaprogram. DSL operations become MLIR.

Python / Meta

DSL / Staged

EXECUTES WHILE GENERATING

Objects, dictionaries, functions, and configuration use Python semantics.



compile

CAPTURED IN MLIR

Typed proxy values and builders describe work for the generated program.

Binding time decides what runs now and what runs in the generated program.

Agenda

From a reusable core to your own dialect.

- 01 BaseDSL, plugins, and decorators
- 02 Types, metaprogramming, and runtime services
- 03 Control-flow capture and worked examples
- 04 Amir: advanced mutation tracking
- 05 Guray: build a Tile IR sub-DSL

Start with a BaseDSL subclass

Give your DSL an identity and a compiler provider.

Sub-DSL setup · target_config defined in notes

```
class MySubDSL(m.BaseDSL):
    plugins = [MyDialectPlugin()]

    def __init__(self):
        super().__init__(
            name="MY_DSL",
            dsl_package_name=["mydsl"],
            **target_config)
```

A subclass chooses policy; BaseDSL supplies shared infrastructure.

Inherit the shared machinery

Capture, specialization,
compilation, and invocation.

Choose target policy once

Compiler, passes, value adapters,
and control-flow builders.

What BaseDSL gives your sub-DSL

Six services that should not be rebuilt for every dialect.

Service	What your DSL supplies or receives
Name	A DSL identity and specialization namespace
Environment	Settings under your prefix, e.g. MY_DSL_PRINT_IR
Dialect plugins	Registration, passes, libraries, and extension hooks
Pass pipeline	Explicit options + environment + plugin passes
JIT compile / execute	Trace, compile, marshal arguments, invoke
Name mangling	Symbols specialized by known arguments

Keep the service classes; move target-specific defaults behind providers and plugins.

Environment settings are reusable infrastructure

Give each DSL its own prefix and make compile-affecting settings explicit.

The extracted environment manager

```
class MyEnv(EnvironmentVarManager):  
    fast_math: bool = env_var(  
        "FAST_MATH", default=False,  
        affects_compile=True)  
  
class MySubDSL(m.BaseDSL):  
    _env_class = MyEnv  
    # name="MY_DSL" in its constructor
```

One namespace per DSL

```
MY_DSL_PRINT_IR  
MY_DSL_CACHE_DIR
```

Correct cache identity

Compile-affecting values participate in specialization.

Reuse the manager; keep GPU architecture and device-specific defaults in the sub-DSL.

Add your downstream dialect with a plugin

Package DkgDSL's extension points as an explicit plugin lifecycle.

Downstream plugin · implementation excerpt

```
class MyDialectPlugin(m.DialectPlugin):
    name = "mydialect"
    def register_dialects(self, context):
        mydialect.register_dialect(context)
    def pipeline_passes(self):
        return ["convert-mydialect-to-llvm"]

class MySubDSL(m.MlirDSL):
    plugins = [MyDialectPlugin()]
```

Your plugin supplies the dialect-specific pieces.

Dialect registration

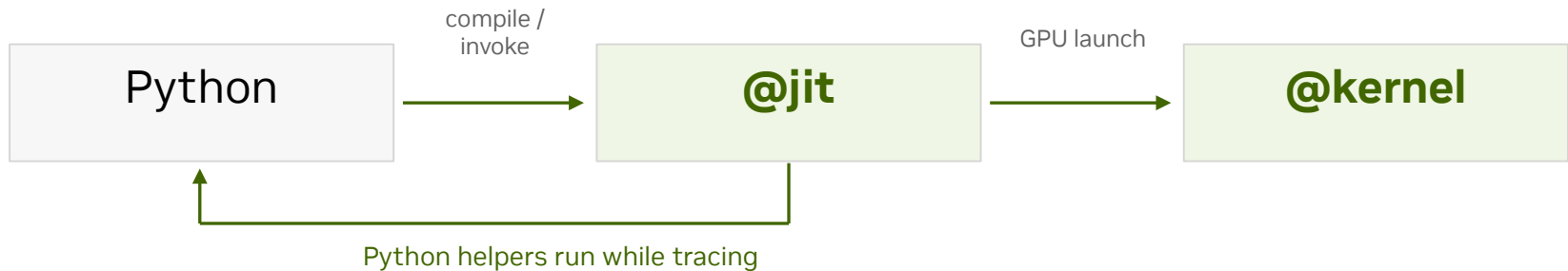
Use bindings built against the same MLIR runtime.

Passes and libraries

Provide your lowering passes and runtime dependencies.

Two decorators; the target chooses how to launch

@jit is general; @kernel describes a GPU kernel.



Python → @jit

Specialize, compile, and execute the host entry.

@jit → @kernel

Capture a device kernel and its GPU launch.

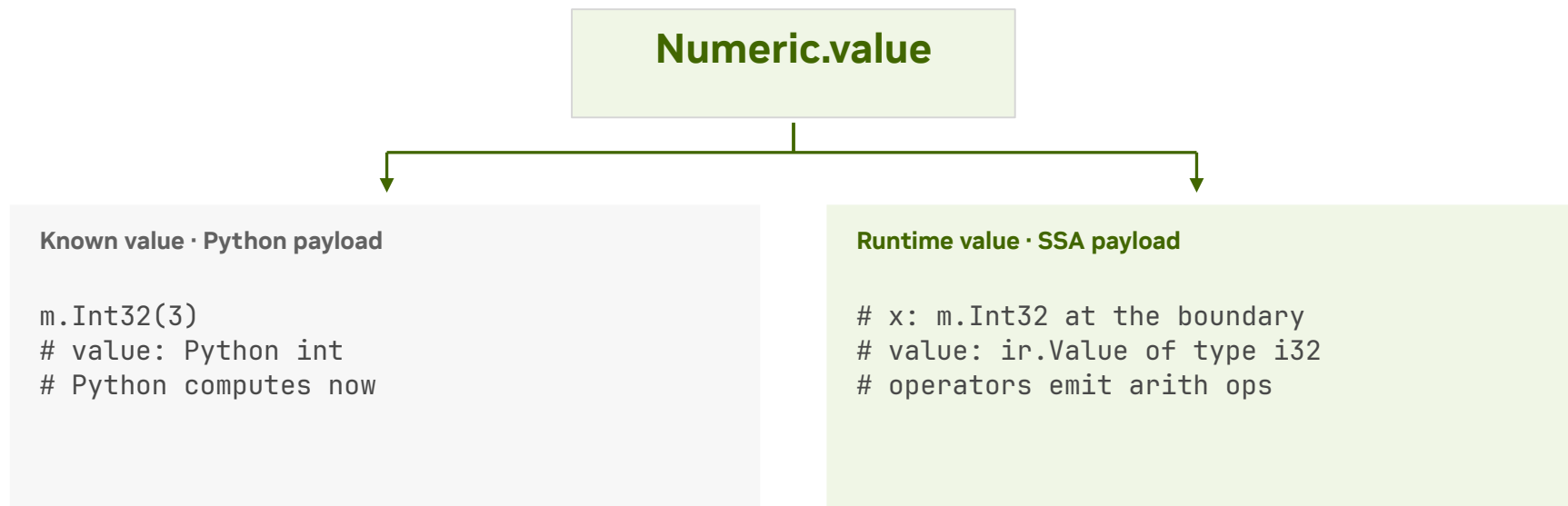
Helpers inline; targets choose launch

Nested helpers run during tracing. Tile IR also supports Python → @kernel directly.

The familiar host path is Python → @jit → @kernel; device-only sub-DSLs can launch directly.

Basic types: builtin types, arith operations

One wrapper can hold a known Python value or an MLIR SSA value.



The Python class carries type policy; the SSA value carries the MLIR type.

A small set of reusable value building blocks

Keep the common type machinery; extend the domain-specific values.

Building block	What the extracted core provides
Numeric types	Int / Float / Boolean; promotion, cast, align, dtype
Pointer	LLVM pointer; load/store, masked access, address arithmetic
Struct + frozen records	Immutable layouts and stable staged state
Vector	A value leaf with splat, sum, and lane access
Interop + math	Lazy Torch dtype bridge; a focused math helper set

Core types compose with registered leaves such as `mydialect.Tensor`.

How type inference works

Types follow annotations, operands, and MLIR results.

An annotated boundary + local inference

```
@m.jit
```

```
def affine(x: m.Int32, factor):  
    return x * factor + 1
```

```
affine(4, 3)
```

```
# x          → runtime i32  
# factor     → known Python int  
# result     → i32
```

At the boundary

Annotations select runtime types; plain configuration can specialize.

At each operator

Promote operand types, materialize constants, and wrap the IR result.

At a control-flow join

Carry the same leaf structure and stable MLIR types.

Inference is local to typed values; it is not whole-program Python type inference.

Use your dialect's arithmetic

Keep numeric policy separate from operation emission.

Downstream value · extension sketch

```
class MyNumber:
    def __init__(self, value):
        self.value = value
    def __add__(self, rhs):
        op = myarith.AddOp(
            self.value, rhs.value)
        return MyNumber(op.result)
```

Dialect plugin · extension sketch

```
class MyArithPlugin(m.DialectPlugin):
    def register_dialects(self, context):
        myarith.register_dialect(context)
    def pipeline_passes(self):
        return ["convert-myarith-to-llvm"]
```

Python operators select arith or your dialect's operations; each value type owns its policy.

Extend the type system: mydialect.Tensor

Package DkgDSL's extract/rebuild value protocol as registered leaves.

Register a downstream Tensor wrapper

```
class Tensor:
    def __init__(self, value):
        self.value = value

m.register_leaf(
    Tensor,
    ir_types=lambda t: [t.value.type],
    ir_values=lambda t: [t.value],
    from_ir_values=lambda old, values:
        Tensor(values[0]))
```

A new dialect type can reuse the same staging and control-flow machinery.

Flatten / rebuild

Carry the tensor's SSA value through calls and regions.

Separate the runtime ABI

Add an argument adapter when host objects cross into compiled code.

Metaprogramming: specialize with Python values

Python configuration becomes constants in the generated program.

Python · executed example

```
@m.jit
def affine(x: m.Int32, scale, bias):
    return x * scale + bias

affine(7, 3, 4) # 25
```

Generated MLIR · body excerpt

```
%c3 = arith.constant 3 : i32
%c4 = arith.constant 4 : i32
%a = arith.muli %x, %c3 : i32
%r = arith.addi %a, %c4 : i32
llvm.return %r : i32
```

scale and bias run in the metaprogram; x remains a runtime argument.

Metaprogramming: objects and dictionaries

Python chooses the operations; the generated program contains their arithmetic.

Ordinary Python polymorphism

```
class Op:
    def apply(self, x):
        raise NotImplementedError

class AddN(Op):
    def __init__(self, n): self.n = n
    def apply(self, x): return x + self.n

class MulN(Op):
    def __init__(self, k): self.k = k
    def apply(self, x): return x * self.k
```

The metaprogram · executed example

```
@m.jit
def pipeline(x: m.Int32, ops: m.Constexpr):
    for op in ops.values():
        x = op.apply(x)
    return x

ops = {"shift": AddN(2), "scale": MulN(3)}
pipeline(4, ops) # 18

# Captured: (x + 2) * 3
```

No runtime dictionary lookup or virtual dispatch is needed for these known objects.

The same metaprogram can build a GPU body

Keep Python object dispatch; capture loads, arithmetic, and stores.

Extracted Array interface · GPU body

```
@m.kernel
def fused(arr: m.Array,
          ops: m.Constexpr):
    tx, _, _ = m.thread_idx()
    x = arr[tx]
    for op in ops.values():
        x = op.apply(x)
    arr[tx] = x

# In @m.jit:
# fused(arr, ops).launch(grid=..., block=...)
```

Meta work

Iterating ops and selecting
AddN.apply / MulN.apply.

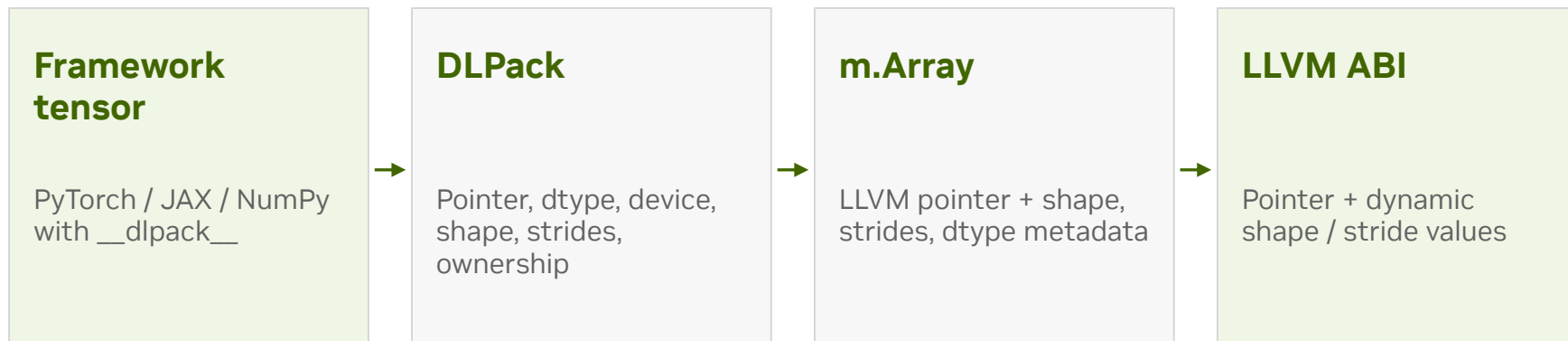
Captured work

Thread index, load, add,
multiply, and store.

The abstraction lives in Python; the generated kernel contains the selected operations.

Carry over Array and the DLPack boundary

PyTorch, JAX, or NumPy storage → a typed view for generated code.



Extraction target: retain the view and ownership contract; separate device allocation and stream policy.

Generate loads and stores while preserving the tensor's layout and lifetime.

Carry over the compile and AOT APIs

CompileCallable separates specialization, execution, and artifact export.

Extracted facade · DkgDSL capability

```
compiled = m.compile[m.GenerateLineInfo(True)](  
    launch, a_sig, b_sig, out_sig)  
  
compiled(a, b, out)  
compiled.export_to_c("build", "vector_add")  
  
# IR for a recipe consumer:  
artifact = m.compile.to_precompiled_mlr(  
    recipe, *signature)
```

Compile a signature

Use representative or fake values;
select options and target artifacts.

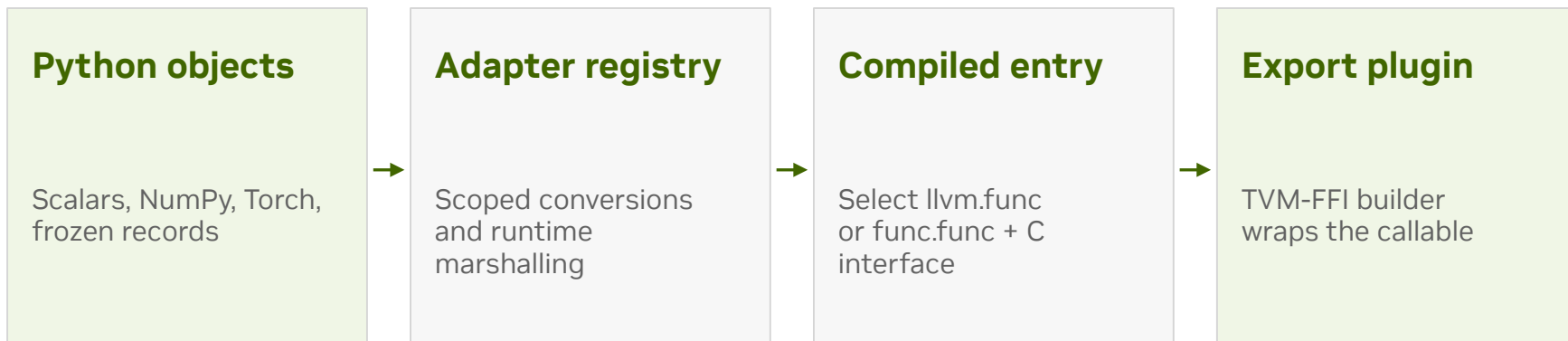
Run or export

Call the executable, emit C artifacts,
or hand IR to another consumer.

Retain the compile API; put target options and artifact formats behind providers.

Reuse argument adapters and export as a plugin

Keep the Python signature, the runtime ABI, and exported entry points connected.



MLIR_DSL_ENABLE_TVM_FFI=1 · optional when the TVM-FFI dependency is available.

Adapters describe data; plugins attach and wrap the compiled entry.

Errors and remarks should explain the source

Connect Python locations to MLIR diagnostics and structured remarks.

A staging failure with an actionable fix

```
acc = 0
for i in range(n):
    acc += i

# error[PHASE_MUTATE_PYTHON]
# `acc` is a Python value.
# Create a runtime value:
# acc = m.Int32(0)
```

Source-located errors

Python file, line, code frame,
and a suggested correction.

MLIR RemarkEngine

Collect analysis and pass remarks;
render them or write a remark file.

MLIR_DSL_DEBUGINFO=1 · MLIR_DSL_REMARKS=.*

Keep compiler work visible

The same tooling should work for every sub-DSL.

Tool	What the user sees
Source locations	Python line → MLIR location → source error / remark
Trace verification	VERIFY_TRACE catches malformed operations near the source
Remark stream	REMARKS / REMARKS_OUTPUT select and serialize records
IR inspection	PRINT_IR / KEEP_IR preserve generated programs
Compile profiling	PROFILE_COMPILER separates tracing, lowering, and execution setup

Extract the source mapping, diagnostics, and profiler alongside compilation.

JIT caching: in memory and on disk

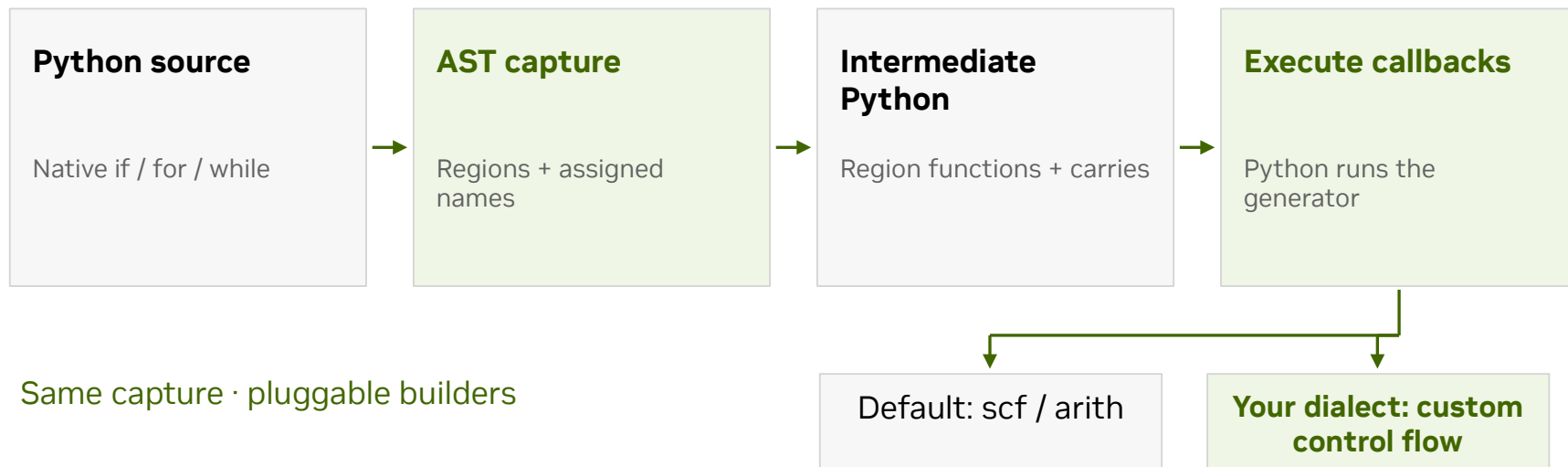
Trace to identify the program; reuse the expensive compilation work.

Layer	Stored value	On a hit
In memory	Compiled function + engine	Reuse the callable
On disk	Lowered MLIR bytecode	Skip passes; recreate the engine
Cache identity	IR + compile settings + toolchain	Separate incompatible programs

Extract cache policy once; let each backend own its compiled artifact and loading.

Control flow: capture the structure with Python's AST

Operator overloading cannot capture Python's native if, for, and while by itself.



Use the AST for control-flow structure; keep Python for the metaprogram.

From an assigned name to an iter_arg

The AST finds the carried name; tracing supplies its SSA value.

Python · symbolic analysis

```
acc = m.Int32(0)
for i in range(n):
    acc = acc + i

# Defined outside: acc
# Written inside:  acc
# write_args = ["acc"]
```

MLIR · region argument and result

```
%sum = scf.for %i = %c0 to %n
    step %c1 iter_args(%acc = %zero)
    → (i32) : i32 {
    %next = arith.addi %acc, %i : i32
    scf.yield %next : i32
}
```

Flatten the carried value, build the region, yield the update, rebuild the result.

Start with immutable staged state

Rebind a new immutable record when its SSA fields change.

A supported Python record

```
from dataclasses import dataclass, replace

@dataclass(frozen=True)
class State:
    total: m.Int32
```

Reconstruct it across the loop

```
@m.jit
def sum_record(n: m.Int32):
    state = State(m.Int32(0))
    for i in range(n):
        state = replace(state,
                        total=state.total + i)
    return state.total

sum_record(10)  # 45
```

A simple first model: frozen records and registered leaves. Amir adds richer mutation tracking.

The same capture can emit your control-flow dialect

Replace the callbacks that build regions and thread their carried values.

Downstream callback installation · sketch

```
class MyControlFlow(m.DialectPlugin):
    def install(self, dsl):
        super().install(dsl)
        dsl.executor.set_functions(
            **mydialect_executors)

class MySubDSL(m.BaseDSL):
    plugins = [MyControlFlow()]
    # Configure BaseDSL as before.
```

Default implementation

Loop / branch callbacks
emit `scf.for`, `scf.if`, `scf.while`.

Your implementation

The same callbacks can emit
`yourdialect.cf` regions and yields.

Reuse AST analysis and the carry protocol; change the region-building callbacks.

Native syntax and explicit builders share the core

Expose region construction even when AST preprocessing is disabled.

Explicit structured control flow

```
@m.jit(preprocess=False)
def sum_upto(n: m.Int32):
    acc = m.Int32(0)
    for i, current, result in m.for_(
        0, n, 1, [acc]):
        m.yield_([current + i])
    return result
```

Region builders

m.for_ / m.if_ / m.while_
with m.yield_.

Boolean helpers

and_ / or_ / not_ / any_ / all_
with dialect-aware comparisons.

Move reusable builders out of the original target DSL and into the shared core.

Worked example: Python → MLIR → execution

A runtime trip count and an SSA accumulator.

Python · CPU example

```
@m.jit
def sum_upto(n: m.Int32):
    acc = m.Int32(0)
    for i in range(n):
        acc += i
    return acc

sum_upto(10)  # 45
```

Generated MLIR · core loop

```
%sum = scf.for %i = %c0 to %n
    step %c1 iter_args(%a = %zero)
    → (i32) : i32 {
    %next = arith.addi %a, %i : i32
    scf.yield %next : i32
}
llvm.return %sum : i32
```

Trace to IR → lower SCF / arith to LLVM → execute → 45.

Worked example: fuse a Python recipe

A known dictionary specializes arithmetic inside a runtime loop.

Python · executed with a NumPy buffer

```
@m.jit
def fused(arr: m.Pointer[m.Float32],
          ops: m.Constexpr):
    for i in m.range(4):
        x = arr[i]
        for op in ops.values():
            x = op.apply(x)
        arr[i] = x

# [0, 1, 2, 3] → [6, 9, 12, 15]
```

Inside the emitted scf.for · excerpt

```
%p = llvm.getelementptr %arr[%i]
      : (!llvm.ptr, i32) → !llvm.ptr, f32
%x = llvm.load %p : !llvm.ptr → f32
%a = arith.addf %x, %two : f32
%r = arith.mulf %a, %three : f32
llvm.store %r, %p : f32, !llvm.ptr
scf.yield
```

The array loop is staged. The object dispatch and recipe loop disappear during generation.

From SSA values to richer mutation

How do we preserve object mutation and aliasing across staged control flow?

— Guray Ozen

Advanced mutation tracking

Reserved for Amir's slides on the PyIR idea.

— Amir Tavakkoli

Tile IR: an existing BaseDSL target

1 / 5 · Extract the common frontend; retain the target implementation.

Existing DKG backend · abridged

```
class CudaTileIR(BaseDSL):
    _env_class = CudaTileIREnvironmentManager
    def __init__(self):
        super().__init__(
            name="CUDA_TILE_DSL",
            dsl_package_name=["tile_ir"],
            compiler_provider=compiler_provider,
            pass_sm_arch_name="compute-capability",
            device_compilation_only=True,
            preprocess=True)
```

DkgDSL already has this target. Move its common dependency to mlir dsl.

Shared machinery

AST capture, staging, diagnostics, environment, and name mangling.

Target choices

Tile values and entry ops, serialization, codegen, launch.

The target supplies the region builder

2 / 5 · The carried-value protocol survives a change of dialect.

Existing Tile IR loop callback · simplified names

```
def build_loop():
    def body(iv, *carried):
        updated = captured_body(iv, *carried)
        tile.loop_continue(tuple(updated))

    return tile.for_loop(
        body, lower_bound=start,
        upper_bound=stop, step=step,
        init_values=list(write_args))
```

Shared capture

Native loop + assigned names
become a body and write_args.

Tile IR generation

cuda_tile.for + continue
replace scf.for + yield.

The same capture protocol can produce a different dialect's regions.

One Python kernel, a Tile IR loop

3 / 5 · Existing DKG API; the shared base is what we extract.

Source-grounded kernel example

```
import tile_ir.cuda_tile_dsl as tile

@tile.kernel
def sum_upto(n: tile.IRValue[tile.Int32]):
    acc = tile.constant(0)
    for i in tile.range_dynamic(n):
        acc += i
    tile.print_tko("sum=%i\n", [acc])
```

Tile IR · loop excerpt

```
%sum = for %i in (%zero to %n,
    step %one) : tile<i32>
    iter_values(%acc = %zero)
    → (tile<i32>) {
    %next = addi %acc, %i : tile<i32>
    continue %next : tile<i32>
}
```

Python runs the generator; Tile IR carries the runtime value.

Compile through the target's toolchain

4 / 5 · Shared lifecycle, target serialization and code generation.

Two implemented backend routes · abridged

```
# Offline path: Tile IR bytecode → cubin
writeBytecode(output, module.operation)
cubin = self._tileiras_compile_cubin(path)

# Alternative: CUDA driver JIT
library = cuda_helpers.load_library_from_file(path)
kernel = cuda_helpers.get_library_kernel(
    library, name)
```

Offline compiler

Serialize Tile IR bytecode;
run tileiras for the target GPU.

Driver JIT

Load Tile IR bytecode through
the CUDA driver library API.

Generation is shared; serialization and device code generation are target policy.

Direct @kernel calls are a target choice

5 / 5 · CUDA Python driver helpers behind a normal Python call.

Existing CUDA launch path · abridged

```
module = cuda_helpers.load_cubin_module(cubin)
kernel = cuda_helpers.get_kernel_function(
    module, name)
cuda_helpers.launch_kernel(
    kernel, num_tiles, (1, 1, 1), stream,
    smem_size=0, kernel_args=packed_args)
```

User-facing call

```
sum_upto(10,
        num_tiles=[1, 1, 1])
```

Expected output

```
sum=45
No host @jit wrapper is required.
```

Extract the machinery once. Each sub-DSL chooses its dialect, compiler, and runtime.